

PROF. DR.-ING. MARK SCHUTERA

DISCIPLINED VIBE

UNFINISHED LECTURE NOTES

Copyright © 2026 Prof. Dr.-Ing. Mark Schutera

PUBLISHED BY UNFINISHED LECTURE NOTES

Combobulated with the help of multiple large language model driven tools. Licensed under the Creative Commons Attribution-NonCommercial 4.0 International License ("CC BY-NC-SA 4.0"). You may not use this file for commercial purposes. If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original You must obtain explicit permission from the author for uses beyond those permitted by this license. To view a copy of this license, visit <https://creativecommons.org/licenses/by-nc-sa/4.0/>. Unless required by applicable law or agreed to in writing, distributed material is provided on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the license for details.

These notes are, by their very nature, unfinished, and they improve with every reader. If you spot an error, disagree with a framing, or want to add a source, a question, or a position, open a pull request at <https://github.com/Quillstacks/LectureMaterial/tree/main/lecturenotes/>. Every contribution is welcome.



*THERE'S A NEW KIND OF CODING I CALL "VIBE CODING", WHERE YOU FULLY
GIVE IN TO THE VIBES, EMBRACE EXPONENTIALS, AND FORGET THAT THE
CODE EVEN EXISTS.*

ANDREJ KARPATHY, 2025

WIR WOLLEN OFFENSIV SPIELEN, ABER MIT HÖCHSTER DISZIPLIN.

JOACHIM LÖW, 2010

Contents

<i>Foundations and the Local Workbench</i>	9
<i>Planning and Design Skills</i>	13
<i>Development Skills</i>	15
<i>Tooling, Guardrails, and Authoring Your Own Skill</i>	19
<i>Closing Notes</i>	23
<i>Index</i>	25

Introduction

These notes accompany a four-block seminar on Disciplined Vibe Coding: programming in which the developer steers a Large Language Model in natural language while the model writes, edits, and reasons about the code. We take Karpathy's framing¹ as the starting point but ground it in concrete craft: the SKILLS catalogue maintained by Matt Pocock² supplies a working vocabulary of reusable agent behaviours, from to-prd to tdd to write-a-skill.

THE STRUCTURE is deliberately interleaved. Each of the four blocks opens with a short theoretical introduction and closes with a hands-on breakout in which students operate a local model (OLLAMA) through the opencode terminal client inside an editor of their choice.

THESE NOTES ARE A LIVING THING, subject to continuous updates. They favour breadth and orientation over completeness; the breakouts are where the actual learning happens.

¹ Coined by Andrej Karpathy in February 2025 and building on his earlier essays Software 2.0 and Software 3.0.

A. Karpathy. There's a new kind of coding i call "vibe coding", 2025b. URL <https://x.com/karpathy/status/1886192184808149383>. Tweet, February 2025

² M. Pocock. skills: A catalogue of claude agent skills, 2025. URL <https://github.com/mattpocock/skills>

Foundations and the Local Workbench

What Vibe Coding Actually Is

KARPATY'S COINAGE describes a shift in the locus of authorship. In Software 1.0, humans wrote explicit instructions; in Software 2.0, gradient descent wrote the weights of neural networks; in Software 3.0³ the program is a natural-language prompt, and the LLM compiles it to behaviour.

“The hottest new programming language is English.”, Andrej Karpathy

VIBE CODING is the working style that follows from this shift. The developer states intent, accepts diffs, runs the result, and iterates without necessarily reading every line. Simon Willison has argued that the term should be reserved for cases where the code is genuinely not read by the human, and that most professional AI-assisted development is something more disciplined.⁴

This seminar treats vibe coding as a spectrum, parameterised by how much the human inspects, structures, and constrains the model's output. SKILLS are the means by which we move along that spectrum deliberately rather than by accident, hence the qualifier disciplined.

A USEFUL COUNTERWEIGHT to the exponentials rhetoric is Fred Brooks's No Silver Bullet⁵: the essential complexity of software, specification, design, verification, is irreducible, and no tool eliminates it. LLMS attack accidental complexity (typing, boilerplate, look-up); the essential part still belongs to you. The skills catalogue is, in effect, a set of disciplines for keeping the essential work in human hands while delegating the accidental work to the model.

Skills as Reusable Prompt Modules

A SKILL, in the Pocock sense, is a small, named, reusable bundle of instructions and examples that an agent loads on demand, a “cheat

³ Karpathy's Software is Changing (Again) talk frames LLMS as a new computing substrate, “LLMs are the new operating system”, with prompts as programs and context windows as RAM.

A. Karpathy. Software is changing (again), 2025a. URL <https://www.youtube.com/watch?v=LCemiRjPEtQ>. Talk, AI Engineer Summit

⁴ S. Willison. Not all ai-assisted programming is vibe coding, 2025. URL <https://simonwillison.net/2025/Mar/19/vibe-coding/>

⁵ F. P. Brooks. No silver bullet—essence and accidents of software engineering. *Computer*, 20(4):10–19, 1987

sheet” in Yegge’s terms⁶, a “recipe” in Crawshaw’s.⁷ The underlying principle is the oldest one in the Pragmatic Programmer’s catalogue⁸: DRY, Don’t Repeat Yourself. A prompt you have written twice is a skill waiting to be extracted.

SKILLS ARE NOT THE ONLY composable layer. The Model Context Protocol⁹ (MCP) provides the complementary half: where a skill is a prompt module that shapes how the agent thinks, an MCP server is a tool surface that determines what the agent can reach, a database, a ticket system, a filesystem, a browser. Skills and MCPs are orthogonal; a mature workbench uses both.

⁶ S. Yegge. Cheating is all you need, 2024. URL <https://sourcegraph.com/blog/cheating-is-all-you-need>

⁷ D. Crawshaw. How i program with llms, 2024. URL <https://crawshaw.io/blog/programming-with-llms>

⁸ A. Hunt and D. Thomas. *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley, 1999

⁹ MCP is an open standard introduced by Anthropic in late 2024 for connecting LLM agents to external data sources and tools through a uniform interface.

Anthropic. Model context protocol, 2024. URL <https://modelcontextprotocol.io>

Layer	Artifact	Example
Model	Weights	Llama 3.2, Qwen 2.5 Coder
Harness	Agent loop, permissions	opencode, Claude Code
MCP server	External tool surface	Filesystem, GitHub, Postgres
Skill	Prompt module	tdd, to-prd
Project	Repository, CLAUDE.md	Your codebase
Intent	The user	You

Table 1: The six layers of a vibe-coding stack.

Breakout 1: Setting Up the Local Workbench

THE GOAL of this breakout is a fully local, offline-capable vibe-coding environment: an IDE, a local model served by OLLAMA, and the opencode terminal client wired to it.

Step 1: Install an Editor

Vibe coding can in principle be done from a bare terminal, but an editor that renders diffs and shows the file tree makes inspection radically easier.

macOS

```
brew install micro
```

Linux (Debian/Ubuntu)

```
sudo apt-get install micro
```

Step 2: Install Ollama and Pull a Coding Model

OLLAMA¹⁰ is a local model runner that exposes an HTTP API compatible with much of the OpenAI surface. On a machine with ≥ 16 GB of

The default in our bootstrap script is `micro`, a ~ 10 MB terminal editor with mouse support and no learning curve, `Ctrl-S` to save, `Ctrl-Q` to quit. It keeps the whole workbench (editor, opencode, shell) in one window. Heavier alternatives, `Zed`, `Sublime`, `vs code`, `JetBrains`, all work equally well; the point is: you should be able to see what the agent changes.

The companion `setup/bootstrap.sh` script in the lecture repository performs all four steps of this breakout in one shot, with `EDITOR=micro|zed|vscode|none` and `MODEL=...` as override knobs. Re-runs are idempotent.

¹⁰ Ollama. Ollama: Get up and running with large language models locally, 2024. URL <https://ollama.com>

RAM a 7B coder model runs comfortably; on ≥ 32 GB you can host a 14B model.

```
# Install Ollama
brew install ollama          # macOS
curl -fsSL https://ollama.com/install.sh | sh  # Linux
```

```
# Start the server (runs on http://localhost:11434)
ollama serve
```

```
# Pull a coding-tuned model in another terminal
ollama pull qwen2.5-coder:7b
ollama run qwen2.5-coder:7b "write a python fizzbuzz"
```

If qwen2.5-coder is too heavy, fall back to llama3.2:3b or deepseek-coder:6.7b. The skill of choosing the smallest model that does the job is itself part of vibe coding.

¹¹ opencode. opencode: Ai coding agent for the terminal, 2025. URL <https://opencode.ai>

Step 3: Install opencode and Connect It to Ollama

opencode¹¹ is a terminal-native agent harness modelled after Claude Code. It can speak to any OpenAI-compatible endpoint, which is exactly what OLLAMA provides.

```
# Install opencode
curl -fsSL https://opencode.ai/install | bash

# Configure a local provider (~/.config/opencode/config.json)
{
  "provider": {
    "ollama": {
      "npm": "@ai-sdk/openai-compatible",
      "options": { "baseUrl": "http://localhost:11434/v1" },
      "models": { "qwen2.5-coder:7b": { "name": "Qwen 2.5 Coder 7B" } }
    }
  }
}
```

```
# Launch opencode in your project directory
cd ~/projects/playground && opencode
```

Step 4: Smoke Test

Inside opencode, ask: “create a Python script that prints the first ten Fibonacci numbers, then run it.” If you see a diff in your IDE and the output 0 1 1 2 3 5 8 13 21 34 in the terminal, the workbench is live.

...

Planning and Design Skills

Why Plan Before You Vibe

THE SINGLE LARGEST FAILURE MODE of naive vibe coding is premature implementation: the model happily produces 400 lines of code for a problem the user has not yet specified. The planning skills in the Pocock catalogue exist to slow that reflex down.

Addy Osmani has named this pattern the “70-percent problem”: AI-assisted code feels 70% done almost immediately, and the remaining 30%, the part that requires actual specification, becomes disproportionately painful.¹² Eric Evans’s older prescription points to the same root cause: agree on the ubiquitous language before you write code.¹³

The Five Planning Skills

`to-prd` converts an open-ended chat into a Product Requirements Document filed as a GitHub issue. The point is not the document but the forcing function: writing a PRD reveals what you do not yet know.

`grill-me` interviews the user relentlessly about a plan, walking each branch of the decision tree until every fork is either closed or explicitly deferred. This is the conversational analog of Kent Beck’s “make the change easy, then make the easy change”¹⁴.

`design-an-interface` spawns parallel sub-agents that each propose a different shape for the same module, a deliberate exercise in designing it twice. The cost of generating three interfaces with an LLM is minutes; the cost of choosing the wrong one in production is months.

`request-refactor-plan` produces a sequence of atomic commits,

¹² A. Osmani. The 70% problem: Hard truths about ai-assisted coding, 2024. URL <https://addyo.substack.com/p/the-70-problem-hard-truths-about>

¹³ E. Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003

Table 2: Planning skills in the Pocock catalogue.

Skill	Output
<code>to-prd</code>	GitHub issue (PRD)
<code>grill-me</code>	Resolved decision tree
<code>design-an-interface</code>	N parallel designs
<code>request-refactor-plan</code>	Tiny-commit plan
<code>to-issues</code>	Vertical-slice tickets

¹⁴ K. Beck. *Tidy First? A Personal Exercise in Empirical Software Design*. O’Reilly, 2024

each independently reviewable. This maps directly onto Beck's tidying-first discipline, onto Fowler's refactoring catalogue, small, behaviour-preserving transformations under green tests¹⁵, and onto the trunk-based development literature.

to-issues encodes Hunt and Thomas's tracer-bullet doctrine¹⁶: rather than specifying the whole system up front, ship a thin end-to-end slice that exercises every layer, then iterate. Vertical slicing is what makes a PRD survive contact with reality, and it is the failure mode most commonly skipped by an over-eager agent.

Breakout 2: Plan a Feature Without Writing It

THE EXERCISE. Pick a small feature for a personal project, a CLI that converts JSON to Markdown tables, say, and walk it through four skills back-to-back, in this order:

1. grill-me: let the agent interview you until it has no more questions.
2. to-prd: emit the resulting PRD as a GitHub issue (or a local Markdown file if you are offline).
3. design-an-interface: generate three competing API shapes; pick one and write a one-paragraph rationale.
4. to-issues: decompose the chosen design into vertical slices.

...

¹⁵ M. Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 2 edition, 2018

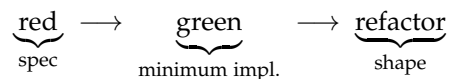
¹⁶ A. Hunt and D. Thomas. *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley, 1999

Constraint: do not let the model write any implementation code during this breakout. The deliverable is a plan you trust, not a running program.

Development Skills

Tests as the Vibe-Coder's Compass

TEST-DRIVEN DEVELOPMENT, due to Kent Beck¹⁷, predates LLMS by two decades but maps onto vibe coding with unusual precision. The red-green-refactor loop gives the agent an unambiguous target, a failing test, and an unambiguous stopping condition, a passing one.



Root-Cause over Symptom

triage-issue encodes the discipline of finding the true cause of a bug before patching its symptom. This is the same medicine that Toyota's five whys prescribes and that SRE post-mortems formalise. Vibe coding without it produces what Crawshaw calls "mole-whacking"¹⁸: each patch creates two new bugs.

MICHAEL FEATHERS gives the operational recipe in Working Effectively with Legacy Code¹⁹: before changing behaviour, write a characterization test that pins down current behaviour, then locate a seam at which the change can be made safely. triage-issue pushes the agent down exactly this path, first reproduce, then isolate, then fix.

Architecture Through Domain Language

improve-codebase-architecture reads two artefacts specifically: a CONTEXT.md that names the domain concepts, and the docs/adr/ folder that records past decisions. Together these are the project's

¹⁷ K. Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2002

This loop also serves as a hallucination filter: the test either passes or it does not. The agent cannot vibe its way past a red bar.

¹⁸ D. Crawshaw. How i program with llms, 2024. URL <https://crawshaw.io/blog/programming-with-llms>

¹⁹ M. C. Feathers. *Working Effectively with Legacy Code*. Prentice Hall, 2004

ubiquitous language and its architectural memory, the two inputs Evans argues are prerequisite to any non-trivial refactor.²⁰

²⁰ E. Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003

JOHN OUSTERHOUT'S *A Philosophy of Software Design*²¹ sharpens the criterion: prefer deep modules, narrow interfaces hiding substantial implementation, over wide, shallow ones. When design-an-interface emits three competing shapes, the question to ask is not "which is prettiest" but "which hides the most complexity behind the smallest surface."

²¹ J. Ousterhout. *A Philosophy of Software Design*. Yaknyam Press, 2018

Skill	When to reach for it	Anchor concept
tdd	New behaviour, fixable bug	Beck, red-green-refactor
triage-issue	Reported defect	Five whys, root cause
improve-...-architecture	Diff friction	Evans, ubiquitous language
scaffold-exercises	Teaching, demos	Worked example pedagogy

Table 3: Development skills mapped to their classical anchors.

Reading Diffs and Resolving Conflicts

INSPECTING WHAT THE AGENT CHANGED is the line between vibe coding and engineering. The interface for that inspection is the *unified diff*, the same format that `git diff` prints and that `opencode` renders before applying any edit. Reading diffs is a prerequisite skill, not an advanced one.

```
--- a/greet.py
+++ b/greet.py
@@ -1,2 +1,2 @@
-def greet(name):
-    return "Hello, " + name + "!"
+def greet(name, lang="en"):
+    return f"Hello, {name}!"
```

The hunk header `@@ -1,2 +1,2 @@` reads as: "the old file's hunk starts at line 1 and is 2 lines long; the new file's hunk starts at line 1 and is 2 lines long." Diffs are read in pairs of `-/+` blocks, not top-to-bottom.

A MERGE CONFLICT happens when two streams of changes touch the same line and GIT cannot decide which to keep. It writes both versions into the file with markers and hands the decision back to the human:

```
<<<<<< HEAD
return f"Hello, {name}!"
=====
greeting = {"en": "Hello", "de": "Hallo"}[lang]
return greeting + ", " + name + "!"
>>>>>> feature/il8n
```


The markers delimit *ours* (HEAD, the branch you are on) and *theirs* (the branch being merged in). Three resolution strategies, in increasing order of judgement required:

1. Take ours wholesale: `git checkout -ours FILE`.
2. Take theirs wholesale: `git checkout -theirs FILE`.
3. Hand-merge: open the file, delete the markers, and write the union that captures both intents.

THE THREE-WAY MERGE is the conceptual model behind `git diff -base`. A clean resolution is a function of three inputs: *ours*, *theirs*, and the common ancestor (*base*). Looking at *ours* versus *theirs* alone is solving the problem with one eye closed; the base tells you which side actually changed what.

```
git diff --ours    FILE    # what you had before the merge
git diff --theirs  FILE    # what is being merged in
git diff --base    FILE    # vs. the common ancestor
git merge --abort          # escape hatch: throws the merge away
```

THREE VIBE-CODING-SPECIFIC FAILURE MODES around diffs: (i) *accepting a hunk without reading the header*, the agent removed a function call three lines above the visible + and you missed it; (ii) *letting the model resolve the conflict for itself*, conflicts encode intent collisions that the model has no way to adjudicate; (iii) *committing with conflict markers still in the file*, which a pre-commit hook would have caught (cf. §).

Breakout 3: Fix a Real Bug, TDD-Style

THE EXERCISE. Take an existing open-source repository (or one of your own) with a known issue. With `opencode` pointed at your local model:

1. Invoke `triage-issue`: have the agent locate the root cause and propose a TDD plan.
2. Invoke `tdd`: write the failing test first, confirm red, then let the agent close it.
3. Inspect the diff before accepting. This is the moment where vibe coding becomes engineering.

...

Most real conflicts demand the third option. The first two are tempting precisely because they are fast, and wrong precisely because they silently discard work, the merge-time analogue of `rm -rf`.

The companion script `setup/diffs_and_merges.sh` builds a sandbox repository at `/tmp/vibe-merge-demo` with a deliberately engineered conflict between an `f-string` refactor and an `i18n` parameter addition. Both changes are correct in isolation; only the hand-merge is correct overall. The accompanying `diffs_and_merges.md` explains every step in prose.

Karpathy's framing reminds us to "embrace exponentials" ; Beck's framing reminds us that an exponential without a test is just a faster way to be wrong.

A. Karpathy. There's a new kind of coding i call "vibe coding", 2025b. URL <https://x.com/karpathy/status/1886192184808149383>. Tweet, February 2025

Tooling, Guardrails, and Authoring Your Own Skill

The Harness Is Part of the Program

A VIBE-CODING SESSION is not just a model and a prompt; it is a harness with hooks, permissions, and a working directory. Every file written, every command run, and every git operation passes through that harness. Configuring it deliberately is the difference between a toy and a tool.

TOOL ACCESS flows through MCP servers attached to the harness. A minimal opencode configuration that wires in a filesystem server and a GitHub server looks as follows:

```
# ~/.config/opencode/config.json
{
  "mcp": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "."]
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_TOKEN": "${env:GITHUB_TOKEN}" }
    }
  }
}
```

Add only the MCP servers you actually need. Each server expands the agent's blast radius; an unused shell-exec MCP is a liability, not a capability.

Pre-Commit Hooks and Git Guardrails

setup-pre-commit installs Husky with lint-staged, type-checking, and tests. The agent now cannot land a commit that breaks the build, a hard constraint that prevents most “70-percent” regressions before they enter GIT history.

`git-guardrails-claude-code` goes further and blocks genuinely destructive commands (`push -force`, `reset -hard`, `branch -D`) at the harness layer. Willison’s writing on prompt injection²² is the conceptual backdrop: an agent that can be steered by untrusted input must not also have unrestricted access to your remote.

²² S. Willison. Prompt injection and jailbreaking are not the same thing, 2024. URL <https://simonwillison.net/2024/Mar/5/prompt-injection-jailbreaking/>

Writing Your Own Skill

`write-a-skill` is the meta-skill. It enforces progressive disclosure: the skill’s top-level description must be short enough to fit in the agent’s selection prompt, while the body and bundled resources are loaded only when the skill is actually invoked. This is the same design principle as Unix man pages, and the reason a catalogue of dozens of skills can coexist without flooding the context window.

THE GRANULARITY RULE for writing a good skill mirrors Robert C. Martin’s rule for writing a good function in Clean Code²³: it should do one thing, do it well, and be nameable in a verb-phrase. `tdd` passes; `do-stuff-with-tests` does not. McConnell’s Code Complete²⁴ adds the missing ingredient: a checklist. Pre-commit hooks and `git` guardrails are checklists rendered executable, the harness will not let the agent forget what you would otherwise forget under pressure.

²³ R. C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008

²⁴ S. McConnell. *Code Complete: A Practical Handbook of Software Construction*. Microsoft Press, 2 edition, 2004

```
,
name: my-domain-glossary
description: Extract a DDD-style ubiquitous-language glossary from
            the current conversation and write it to CONTEXT.md.
,
```

Body

1. Scan the current transcript for domain nouns and verbs.
2. Group them by aggregate.
3. Emit a Markdown table to `CONTEXT.md`.

Breakout 4: Author and Use Your Own Skill

THE EXERCISE. Identify one task you have done by hand at least three times this term, tagging exam questions, summarising seminar notes, drafting issue triage comments, and convert it into a skill.

1. Run `write-a-skill` and let the agent scaffold the directory layout.

2. Edit the description until it would unambiguously match your task and not someone else's.
3. Install it (`~/ .config/opcode/skills/` or the equivalent for your harness).
4. Invoke it on a fresh instance of the task. Iterate until the output is acceptable without manual edits.
5. Configure `setup-pre-commit` and `git-guardrails-claude-code` on the same project so the skill cannot regress your repository.

...

The deliverable is a .zip of your skill folder plus a one-page reflection: what was hard to specify, and why?

Closing Notes

VIBE CODING is neither magic nor a substitute for engineering judgement. It is a new interface to the same craft, and like any interface it rewards practitioners who understand the layers underneath. The four blocks of this seminar trace those layers from the model on disk to the skill in your editor; the breakouts are where the understanding actually lands.

“AI-assisted programming will not replace the deliberate practice of software engineering. It will, however, make the practitioners who refuse to develop that practice obsolete much faster.”, paraphrasing Crawshaw²⁵

²⁵ D. Crawshaw. How i program with llms, 2024. URL <https://crawshaw.io/blog/programming-with-llms>

Index

70-percent problem, [13](#)

Brooks

 No Silver Bullet, [9](#)

characterization tests, [15](#)

checklists, [20](#)

Clean Code, [20](#)

Code Complete, [20](#)

DDD, [13](#)

deep modules, [16](#)

design-an-interface, [13](#)

diff, [16](#)

DRY, [10](#)

essential complexity, [9](#)

Feathers, [15](#)

Fowler, [14](#)

grill-me, [13](#)

guardrails, [20](#)

harness, [19](#)

hooks, [19](#)

Karpathy, [9](#)

license, [2](#)

MCP, [10](#)

merge conflict, [16](#)

Model Context Protocol, [10](#)

Ousterhout, [16](#)

planning, [13](#)

Pragmatic Programmer, [10](#)

pre-commit, [19](#)

progressive disclosure, [20](#)

prompt engineering, [10](#)

prompt injection, [20](#)

red-green-refactor, [15](#)

refactor, [14](#)

root cause, [15](#)

seams, [15](#)

skills, [10](#)

Software 2.0, [9](#)

Software 3.0, [9](#)

TDD, [15](#)

three-way merge, [17](#)

to-prd, [13](#)

tracer bullets, [14](#)

triage, [15](#)

ubiquitous language, [13](#)

unified diff, [16](#)

vertical slicing, [14](#)

vibe coding

 definition, [9](#)

write-a-skill, [20](#)